



Performance engineering: experiment management, MLflow

Nebius Academy

May 2026

Emeli Dral

Experiment Tracking

Experiment Tracking

Hardest Part of ML isn't ML, it's Data

"Hidden Technical Debt in Machine Learning Systems," Google NIPS 2015

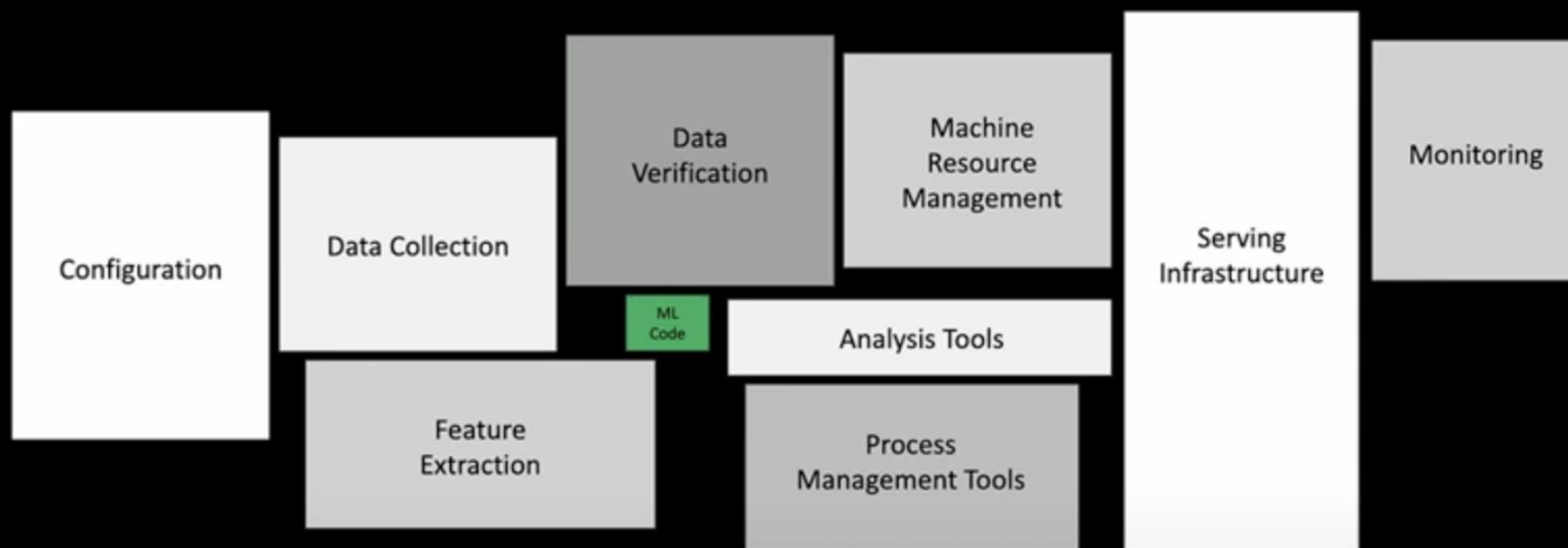


Figure 1: Only a small fraction of real-world ML systems is composed of the ML code, as shown by the small green box in the middle. The required surrounding infrastructure is vast and complex.

Experiment Tracking

Experiments

It is important to keep track of experiments:

- model's parameters
- results of experiments

Experiment Tracking

Experiments

It is important to keep track of experiments:

- model's parameters
- results of experiments

Model ID	Hyper params	PR	RC
01		0.77	0.65
02		0.77	0.66
07		0.68	0.59
...		...	...
34578		0.847	0.74

? Is that enough?

Experiment Tracking

What is experiment tracking?

It is important to keep track of all relevant information related to the experiment:

- source code
- trained model
- model's parameters
- results of experiments
- experiment environment
- data sources and datasets

Experiment Tracking

Why track experiments?

- reproducible research
- well organised teamwork (including a team of one 😊)
- optimal selection of best model configurations

Miflow Overview

MLflow Overview

Deliver High-Quality AI, Fast

Building AI products is all about iteration.

MLflow lets you move 10x faster by simplifying how you

debug, evaluate, and monitor your LLM applications, Agents, and Models.

Try Demo

Get Started



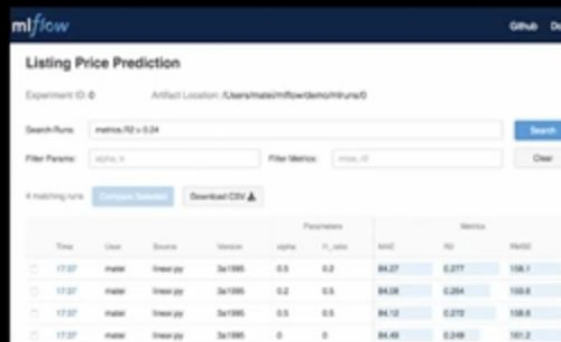
25K+ Stars



30M+ Downloads/mo

Mlflow Overview

What's MLFlow ?



Listing Price Prediction

Experiment ID: 0 Artifact Location: /Users/mlflow/demo/mlruns/0

Search Runs: Search

Filter Params: Filter Metrics: Clear

4 matching runs [Compare](#) [Download CSV](#)

	Time	Run	Source	Version	alpha	rmse	rmse	F2	RMSE
1	11:27	mlflow	train.py	3a1995	0.5	0.2	84.27	0.277	108.1
2	11:27	mlflow	train.py	3a1995	0.2	0.5	84.08	0.284	108.6
3	11:27	mlflow	train.py	3a1995	0.5	0.5	84.15	0.272	108.6
4	11:27	mlflow	train.py	3a1995	0	0	84.43	0.248	101.2

Web UI

Visualize experiments

+

```
import mlflow

# log model's tuning parameters

with mlflow.start_run():
    mlflow.log_param("layers", layers)
    mlflow.log_param("alpha", alpha)
    mlflow.log_artifact("plot", image)
```

API

Track & automate

+

```
mlflow models serve
-m "dbfs:xxx/my_model"
```

CLI

Run & Deploy

Mlflow Overview

Running MLflow for ML Models Anywhere

MLflow can be used in a variety of environments, including your local environment, on-premises clusters, cloud platforms, and managed services. Being an open-source platform, MLflow is **vendor-neutral**; whether you're building AI agents, LLM applications, or ML models, you have access to MLflow's core capabilities — tracing, evaluation, experiment tracking, deployment, and more.



Amazon SageMaker



Azure Machine Learning

NEBIUS



kubernetes

Mlflow Overview

LLMs & Agents

Debug, evaluate, monitor, and optimize your AI agents and LLM applications, with production-grade tracing, evaluation, prompt management, and much more.

Model Training

Manage the full machine learning and deep learning model lifecycle, with experiment tracking, hyperparameter tuning, and beyond.

Mlflow: Model Training

Model Training

MLflow Tracking

Record and query experiments: code, data, config, and results

[Read more](#)

MLflow Projects

Package data science code in a format to reproduce runs on any platform

[Read more](#)

MLflow Models

Deploy machine learning models in diverse serving environments

[Read more](#)

Model Registry

Store, annotate, discover, and manage models in a central repository

[Read more](#)

Model Training

MLflow tracking

MLflow Tracking is organized around the concept of *runs*, which are executions of some piece of data science code. Each run records the following information:

Code Version

Git commit hash used for the run, if it was run from an [MLflow Project](#).

Start & End Time

Start and end time of the run

Source

Name of the file to launch the run, or the project name and entry point for the run if run from an [MLflow Project](#).

Model Training

MLflow Tracking

MLflow Tracking is organized around the concept of *runs*, which are executions of some piece of data science code. Each run records the following information:

Parameters

Key-value input parameters of your choice. Both keys and values are strings.

Metrics

Key-value metrics, where the value is numeric. Each metric can be updated throughout the course of the run (for example, to track how your model's loss function is converging), and MLflow records and lets you visualize the metric's full history.

Artifacts

Output files in any format. For example, you can record images (for example, PNGs), models (for example, a pickled scikit-learn model), and data files (for example, a [Parquet](#) file) as artifacts.

Model Training

MLflow Project

At the core, MLflow Projects are just a convention for organizing and describing your code to let other data scientists (or automated tools) run it. Each project is simply a directory of files, or a Git repository, containing your code.

Model Training

MLflow Project

Name

A human-readable name for the project.

Entry Points

Commands that can be run within the project, and information about their parameters. Most projects contain at least one entry point that you want other users to call. Some projects can also contain more than one entry point: for example, you might have a single Git repository containing multiple featurization algorithms. You can also call any **.py** or **.sh** file in the project as an entry point. If you list your entry points in a **MLproject** file, however, you can also specify *parameters* for them, including data types and default values.

Environment

The software environment that should be used to execute project entry points. This includes all library dependencies required by the project code. See [Project Environments](#) for more information about the software environments supported by MLflow Projects, including [Conda environments](#), [Virtualenv environments](#), and [Docker containers](#).

Model Training

MLflow Models

An MLflow Model is a standard format for packaging machine learning models that can be used in a variety of downstream tools—for example, real-time serving through a REST API or batch inference on Apache Spark.

All of the flavors that a particular model supports are defined in its `MLmodel` file in YAML format. For example, `mlflow.sklearn` outputs models as follows:

```
# Directory written by mlflow.sklearn.save_model(model, "my_model")
my_model/
├── MLmodel
├── model.pkl
├── conda.yaml
├── python_env.yaml
└── requirements.txt
```

Model Training

Mlflow Models

```
time_created: 2018-05-25T17:28:53.35

flavors:
  sklearn:
    sklearn_version: 0.19.1
    pickled_model: model.pkl
  python_function:
    loader_module: mlflow.sklearn
```

This model can then be used with any tool that supports either the `sklearn` or `python_function` model flavor. For example, the `mlflow models serve` command can serve a model with the `python_function` or the `crate` (R Function) flavor:

```
mlflow models serve -m my_model
```

Model Training

MLflow Model Registry

The MLflow Model Registry component is a centralized model store, set of APIs, and UI, to collaboratively manage the full lifecycle of an MLflow Model.

It provides:

- model lineage (e.g. experiment and run produced the model),
- model versioning,
- stage transitions (e.g. staging to production),
- annotations.

Model Training

Mlflow Model Registry

mlflow					GitHub	Docs
Registered Models					search model name	
Name ▴ ▾	Latest Version	Staging	Production	Last Modified ▴ ▾		
Model A	Version 1	Version 1	—	2019-10-16 22:51:19		
Model B	Version 1	—	—	2019-10-16 22:51:52		
					< 1 >	

Model Training

Mlflow Model Registry

Registered Models > Model A > Version 1 ▾

Registered At: 2019-10-17 13:38:51

Creator: 

Last Modified: 2019-11-12 09:56:00

Source Run: [Run b99a0fc567ae4d32994392c800c0b6ce](#)

▾ Description 

Stage: **Staging** ▾

- Transition to → **Staging**
- Transition to → **Production**
- Transition to → Archived

Miflow: LLM & Agents

LLM & Agents

LLM & Agents

MLflow is the largest open source AI engineering platform for agents and LLMs. MLflow enables teams of all sizes to debug, evaluate, monitor, and optimize production-quality AI applications, while controlling costs and managing access to models and data.



All-in-one Platform

Manage the complete AI development journey from prototype to production. Track prompts, evaluate quality, deploy AI agents, and monitor performance in one platform.



Complete Observability

See inside every AI decision with comprehensive tracing that captures prompts, retrievals, tool calls, and LLM responses. Debug complex workflows with confidence.



Evaluation & Monitoring

Stop manual testing with LLM judges and custom metrics. Systematically evaluate every change to ensure consistent improvements in your AI applications.



Framework Integration

Use any agent framework or LLM provider. With 100+ integrations and extensible APIs, MLflow adapts to your tech stack, not the other way around.

LLM & Agents

Observability

MLflow's open-source, OpenTelemetry-compatible tracing captures app's entire execution, including prompts, retrievals and tool calls.

The screenshot displays the MLflow Observability interface for an experiment named "Write a paragraph about CI/CD is applicable in AI". The interface includes a sidebar with navigation options like "Experiments", "Trace", "Last 7 days", "Search", and "Detect Issues". The main panel shows the trace details for a specific run with ID "tr-863c14298a96fcf4b4a5edf64e8888f5". Key metrics displayed are: Status "OK", Token count "229", Cost "\$0.001232", and Latency "2.92s". The trace is divided into sections: "Summary", "Details & Timeline", and "Linked prompts". The "Summary" section shows the "output" of the LLM, which is a paragraph about CI/CD. Below this, the "parallel_tool_calls" section shows the value "true", and the "temperature" section shows the value "1".

Experiments > < > Write a paragraph about CI/CD is applicable in AI

Trace Status OK ID tr-863c14298a96fcf4b4a5edf64e8888f5 Token count 229 Cost \$0.001232 Latency 2.92s

Summary Details & Timeline Linked prompts

Last 7 days

Search

Detect Issues

Trace ID

tr-38d0f

tr-4ea37

tr-67eb3

output

Assistant

Ever wish your AI projects could stop acting like moody teenagers—working perfectly one day, then totally rebelling the next? Enter CI/CD! hype squad, making sure every line of code, every model tweak, and every wild experiment moves seamle...

See more

parallel_tool_calls

true

temperature

1

Evaluations

[Content Generation](#) > [Evaluations](#) >

optimized_tweet_generation

Overview [Model metrics](#) [System metrics](#) [Traces](#) [Artifacts](#)

Description

No description

Metrics (6)

 Search metrics

Metric	Value
avg_length	486.6
length_pass_rate	0
avg_sentiment	0.48368
sentiment_pass_rate	0.4
engaging_share	1
engaging_count	5

LLM & Agents

Prompt Management & Optimization

Create prompt



Name:

Provide an unique prompt name

Prompt type:

Text

Chat

Prompt:

Type prompt content here. Wrap variables with double curly brace e.g. {{ name }}.

Commit message (optional):

› [Advanced settings \(optional\)](#)

Cancel

Create

AI Gateway

AI Gateway provides a unified interface for deploying and managing multiple LLM providers within an organization. Advanced routing capabilities enable traffic splitting for A/B testing and automatic failover chains for high availability.



Unified Interface

Access multiple LLM providers through a single endpoint, eliminating the need to integrate with each provider individually.



Centralized Security

Store LLM provider API keys in one secure location with request/response logging for audit trails and compliance.



Advanced Routing

Traffic splitting for A/B testing and automatic fallbacks ensure high availability across providers.



Zero-Downtime Updates

Add, remove, or modify endpoints dynamically without restarting the server or disrupting running applications.

Performance engineering: experiment management, MLflow

Thank you!
Emeli Dral